src/embeddings.py

Turning text into vectors with a local sentence-transformer

This module converts text into **embeddings** — numeric vectors where similar meanings sit close together — using a small model that runs locally on the CPU. It is the foundation of semantic search: every chunk and every query passes through here.

Walkthrough of all 61 lines, in order. Left numbers are the real line numbers in the file.

```
 1  """Turn text into vectors with a local sentence-transformer.
 2
 3  An *embedding* is the whole game behind semantic retrieval: it maps a piece of
 4  text to a point in a high-dimensional space such that texts about similar things
 5  land near each other. "How do transformers attend to tokens?" and "the attention
 6  mechanism computes weighted sums over the sequence" have almost no words in
 7  common, but their embeddings are close — so a query can find a chunk by *meaning*,
 8  not by keyword overlap.
 9
10  We use `all-MiniLM-L6-v2` (chosen and justified in config.py). It runs on CPU,
11  needs no API key, and is deterministic given a fixed seed — which is what keeps
12  the entire retrieval path reproducible on any machine.
13
14  The model is loaded lazily and cached: importing this module is cheap, and the
15  (~90 MB) model downloads only the first time it is actually used.
16  """
```

Lines 1 to 16 are the module docstring. They explain what an embedding is (a map from text to a point in high-dimensional space where related texts land near each other), give a concrete example of two differently-worded but semantically close sentences, name the chosen model (all-MiniLM-L6-v2), and note two deliberate properties: it runs on CPU with no API key, and it is loaded lazily so importing the module stays cheap.

```
17
18  from __future__ import annotations
19
20  import functools
21
22  import numpy as np
23
24  from . import config
```

Lines 17 to 24 are the imports, grouped by origin. Line 18 imports the future annotations behaviour so type hints like list[str] are treated as strings and never evaluated at runtime. Line 20 imports functools (used for the caching decorator below). Line 22 imports numpy as np, the array library the vectors are returned as. Line 24 imports the project's config module for the model name and batch size. The blank lines separate standard-library from third-party from local imports.

```
25
26
27  @functools.lru_cache(maxsize=1)
```

Lines 25 to 27: two blank lines (PEP 8 spacing before a top-level definition) and then the decorator @functools.lru_cache(maxsize=1). This memoizes the function below so the model is constructed only once per process; every later call returns the same cached object instead of reloading a 90 MB model.

```
28  def get_model():
```

Line 28 defines get_model(), the function that loads the sentence-transformer.

```
29      """Load (and cache) the sentence-transformer.
30
31      Import is inside the function so that merely importing this module — which
32      the tests and the config-reading tools do — does not require
33      sentence-transformers to be installed or the model to be downloaded.
34      """
```

Lines 29 to 34 are the function's docstring, explaining a key design choice: the heavy import lives inside the function, not at the top of the file, so that merely importing this module (as the tests and config tools do) does not require sentence-transformers to be installed or the model to be downloaded.

```
35      from sentence_transformers import SentenceTransformer
```

Line 35 performs that deferred import: it brings in SentenceTransformer only when the model is actually needed.

```
36
37      return SentenceTransformer(config.EMBED_MODEL)
```

Lines 36 to 37: a blank line, then the return statement that constructs the model from the name in config.EMBED_MODEL and hands it back. Thanks to the cache decorator this construction happens at most once.

```
38
39
40  def embed_texts(texts: list[str]) -> np.ndarray:
```

Lines 38 to 40: two blank lines, then the definition of embed_texts, which takes a list of strings and is annotated to return a numpy array.

```
41      """Embed a list of texts into an (n, EMBED_DIM) float32 array.
42
43      Vectors are L2-normalized so that an inner product between two of them equals
44      their cosine similarity. That equivalence is what lets the FAISS index use a
45      fast inner-product search and have it mean "most semantically similar" —
46      see vectorstore.py.
47      """
```

Lines 41 to 47 are the docstring. They state the output shape (n by EMBED_DIM, float32) and the single most important detail: the vectors are L2-normalized so that an inner product between any two of them equals their cosine similarity — the property that lets the FAISS index use fast inner-product search and have it mean most similar.

```
48      model = get_model()
```

Line 48 fetches the cached model by calling get_model().

```
49      vectors = model.encode(
50          texts,
51          batch_size=config.EMBED_BATCH_SIZE,
52          normalize_embeddings=True,      # unit vectors → inner product = cosine
53          show_progress_bar=False,
54          convert_to_numpy=True,
55      )
```

Lines 49 to 55 call model.encode on the input texts. batch_size comes from config; normalize_embeddings=True is what produces the unit vectors described above; show_progress_bar=False keeps output clean; convert_to_numpy=True returns a numpy array rather than a torch tensor.

```
56      return vectors.astype(np.float32)
```

Line 56 casts the result to float32 and returns it. float32 halves memory versus float64 and is the precision FAISS expects.

```
57
58
59  def embed_query(query: str) -> np.ndarray:
```

Lines 57 to 59: two blank lines, then the definition of embed_query, a small convenience wrapper for embedding a single query string.

```
60      """Embed a single query into a (1, EMBED_DIM) array, ready for search."""
```

Line 60 is its one-line docstring, noting it returns a 1-by-EMBED_DIM array shaped ready for the vector store's search method.

```
61      return embed_texts([query])
```

Line 61 returns embed_texts([query]) — it simply wraps the single query in a list and reuses the batch function, guaranteeing a query is embedded exactly the same way as the documents it will be compared against.